

```
# Introduction
TODO: Give a short introduction of your project. Let this section explain the objectives
or the motivation behind this project.

# Getting Started
TODO: Guide users through getting your code up and running on their own system. In this
section you can talk about:
1. Installation process
2. Software dependencies
3. Latest releases
4. API references

# Build and Test
TODO: Describe and show how to build your code and run the tests.

# Contribute
TODO: Explain how other users and developers can contribute to make your code better.

# Install nuget:
1. on your project -> manage nuget packages
2. settings( near package source) -> the green +
3. in source put the URL: \\pnasm8\perry\346_Nugets -> OK
4. in package source -> the name you put in (3) -> search Renassaince.QueueManager->
install

# Important Things:
1. loginData
  1. ems DetailsConnect to Producer:
    @"{"Url":"url wanted","User":"user wanted","Pass":"pass
    wanted","Factory":"Factory wanted","Transacted":false,
    "ClientAck":false,"QueueName":"","Count":0,
    "ThrowAfterDisconnect":true/false}"
    or
    @"{"Url":"url wanted","User":"user wanted","Pass":"pass
    wanted","Factory":"Factory wanted","Transacted":false,"ClientAck":false,
    "QueueName":"","Count":0, "ThrowAfterDisconnect":true/false}"

  2. ems DetailsConnect to Consumer:
    @"{"Url":"url wanted","User":"user wanted","Pass":"pass
    wanted","Factory":"Factory wanted","Transacted":false/true(should be false),
    "ClientAck":false/true(should be true),"QueueName":"QueueName wanted",
    "Count":who many Consumers(should be 1), "ThrowAfterDisconnect":true/false}"

  3. rest DetailsConnect to Producer:
    @"{"Url":"url wanted","Method":"GET/POST"}";

  4. SIGN ADAPTER DetailsConnect to Producer:
    @"{"SignAdapterURL":"url of the SignAdapter given from 349 ","Url":"Queue
    manager url","FactoryName":"Factory wanted","QueueName":"QueueName
    wanted","UserName":"empty or user if you want to do impersonation",
    "UserPassword":"empty or user if you want to do impersonation",
    "UseDip":empty(default as false) or true/false,
    "DoNotUseProjectSignature":empty(default as false) or true/false,
    "IsXmlMessage":empty(default as false) or true/false}"

  5. File DetailsConnect to Producer:
    @"{"Time":0,"FilePath":"" full file path to write to, with file name: like
    C://Dev_2//t1.txt"}";

  6. File DetailsConnect to Consumer:
    @"{"Time":timer in milisec(should be between 1000-10000),"FilePath":"" full
    folder path to write to consume"}";

  7. rabbit DetailsConnect to Producer/Consumer:
    @"{"RabbitHostname":"rabbit port, like nrbtr rabbit is
    50.2.2.78","RabbitPort":5672,"RabbitUsername":"user name for rabbit ",
    "RabbitPassword":"password for rabbit", "RabbitVirtualHost":"VirtualHost
    wanted ", "RabbitRequestedHeartbeatSeconds":should be between 5 and 20
    seconds(send int), "RabbitContinuationTimeout":should be around 60(send int),
```



```
"ExchangeName": "ExchangeName wanted", "RoutingKey": "RoutingKey of the queue",
"QueueName": "QueueName wanted", "Count": who many Consumers(should be 1),
"ThrowAfterDisconnect": true/false}"
```

2. call factory

```
1.ConsumerFactory.CreateConsumer(DataAccess: from proj
dll:DataAccessEnum.EMS/DataAccessEnum.RABBIT...,
string loginData (like 1.1/1.2), string name, log that implaments ILogger).
```

2.exsample:

```
this.consumer = ConsumerFactory.CreateConsumer(DataAccessEnum.EMS,
loginDataConsumer, "EMSLISTENER", _logger);
```

3. if you do not want to use log, put NullLogger.Instance(using Microsoft.Extensions.Logging.Abstractions) in the factory:

like:

```
this.consumer = ConsumerFactory.CreateConsumer(DataAccessEnum.EMS,
loginDataConsumer, "EMSLISTENER", NullLogger.Instance);
```

4. id you want many instances of consumer/producer, give diference name in the factory,

like:

```
this.consumer1 = ConsumerFactory.CreateConsumer(DataAccessEnum.EMS,
loginDataConsumer, "EMSLISTENER", NullLogger.Instance);
this.consumer2 = ConsumerFactory.CreateConsumer(DataAccessEnum.RABBIT,
loginDataConsumer, "RABBITLISTENER", NullLogger.Instance);
```

3. CreateTemporaryDestination:

1. if destination exsist, only listen to the destination.

2. if destination doesnot exsist, create temporary destination and listen to it.

3. *important*- do consumer.Start(); before adding the eventHandler or something else with the consumer(one time is enough, but more is fine)

4. example:

```
consumer.Start(); //start a connection with the queue server
consumer.CreateTemporaryDestination("test.itay.1");
consumer.StartConsumer("queueName", Listener_OnMessage); //start a seesion that
consume to queue
```

```
consumer.Start();//not necessary
consumer.CreateTemporaryDestination("test.itay.2");
consumer.StartConsumer("queueName", Listener_OnMessage);
```

5. example for queue with more than 1 consumer:

```
for (int i = 0; i < consumersCount; i++)
{
    consumer.Start();
    consumer.CreateTemporaryDestination("test.itay.2");
    consumer.StartConsumer("test.itay.2", Listener_OnMessage, i);
}
```

4.IConsumer: StartConsumer

1. StartConsumer(string destName, Action<object, EventArgs> action, int index = 0);
1.destName- destination name to consume(**ems, rabbit: queue name, file: folder path**)
2.action- the method you have when message arrived. like Listener_OnMessage in (3.4).
3.index- used only if you have more than one consumer to queue

2. example if you want to have same number of consumers like Count(1.2\1.7) and consume to the queue(1.2\1.7) you give in the login data(1.2\1.7)

```
consumer.Start();
consumer.StartConsumer(null or "", Listener_OnMessage);
```

2. example for queue that have only 1 consumer:

```
consumer.Start();
consumer.StartConsumer("queueName", Listener_OnMessage);
```

3. example for queue with more than 1 consumer:

```
for (int i = 0; i < consumersCount; i++)
```



```
{
    consumer.Start();
    consumer.StartConsumer("queueName", Listener_OnMessage, i);
}
```

4. example for the Listener_OnMessage eventHandler:

```
1. public void Listener_OnMessage(object sender, EventArgs eventArgs)
{
    var requestDataHolder = (eventArgs as MessageEventArgs);
    bool s = consumer.Acknowledge(requestDataHolder.MessageID);
}
```

2. MessageEventArgs- class that you get from the project's nuget

5. send message

1. producer.Send("message", "applicationProperties");

1. ems:

1. the Destination is what you give in the DetailsConnect to Producer(like 1.1) in QueueName
2. **do not use this metod** if QueueName in (like 1.1) is empty or you just want to send message to other Queue
3. if you do not want to use rbc headers, applicationProperties should be null or empty, else: you need to put xml string that matching rbc xsd.

2. Rest:

1. the Destination is what you give in the DetailsConnect to Producer(like 1.3) in Url
2. **do not use this metod** if url in (like 1.3) is empty or you just want to send message to other url
3. applicationProperties should be null or empty.
4. if in (1.3) method is GET: in the rest server get method: to get the message you need to do:


```
string message = Request.Query.Keys.ElementAt(0);
```
5. if in (1.3) method is POST: you get the message in the body in the rest server post method

3. Rabbit:

1. the Destination is what you give in the DetailsConnect to Producer(like 1.7) in QueueName
2. **do not use this metod** if QueueName in (like 1.7) is empty or you just want to send message to other Queue
3. if destination doesn't exist: return false
4. if you do not want to use rbc headers, applicationProperties should be null or empty, else: you need to put xml string that matching rbc xsd.

2. producer.Send("message", "destination", "applicationProperties");

1. ems:

1. the destination is the ***queue name***.
2. if you do not want to use rbc headers, applicationProperties should be null or empty.
3. if destination doesn't exist: create temporaryQueue and send the message there.

2. Rest:

1. the destination is the ***url for the rest method***.
2. applicationProperties should be null or empty.
3. if in (1.3) method is GET: in the rest server get method: to get the message you need to do:


```
string message = Request.Query.Keys.ElementAt(0);
```
4. if in (1.3) method is POST: you get the message in the body in the rest server post method

1. rabbit:

1. the destination is the ***QueueName*** of the queue.
2. if you do not want to use rbc headers, applicationProperties should be null or empty.
3. if destination doesn't exist: return false

3. producer.Send("message", "destination", "detailsHeaders", "applicationProperties");

1. ems:

1. headers data:detailsHeaders=


```
@{"DeliveryMode":"NonPersistent/PERSISTENT/ReliableDelivery", "Priority":int- more than 1(should be 4), "TimeToLive":experation time of the message in milSec}";
```
2. if you do not want to use rbc headers, applicationProperties should be

- 3. if destination doesn't exist: create temporaryQueue and send the message there.
- 2. rest:
 - 1. like (5.2.2)
- 3. rabbit:
 - 1. headers data:detailsHeaders= @"{"TimeToLive":expiration time of the message in milSec}";
 - 2. if you do not want to use rbc headers, applicationProperties should be null or empty, else: you need to put xml string that matching rbc xsd.]
 - 3. if destination doesn't exist: return false